

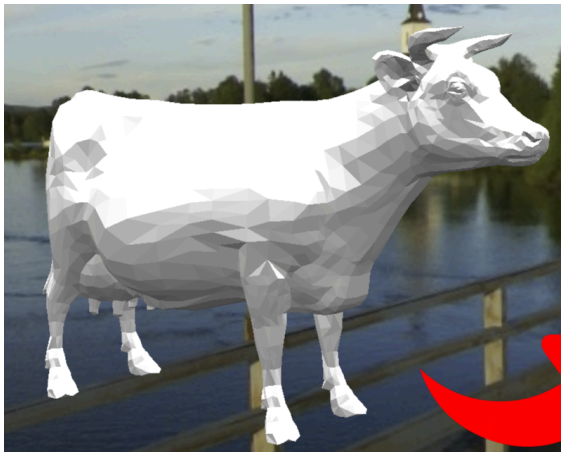
LAPORAN TUCIL_2

IF2211 STRATEGI ALGORITMA | SEMESTER II 2025/2026

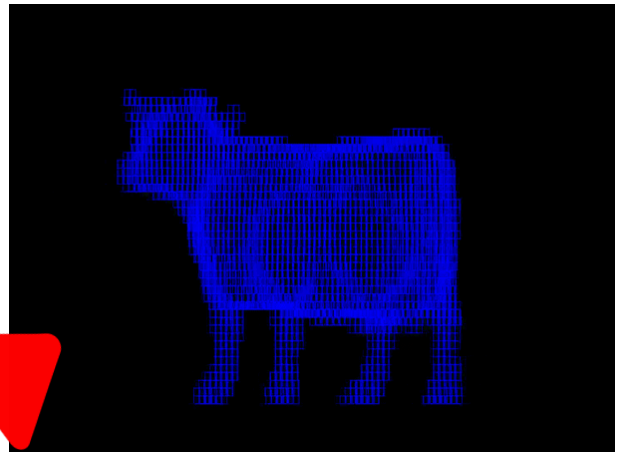
Voxelization objek 3D menggunakan octree

Raysha Erviandika Putra | 13524050 | 13524050@std.stei.itb.ac.id
I Gusti Ngurah Alit Dharma Yudha | 13524072 | 13524072@std.stei.itb.ac.id

I. Latar Belakang

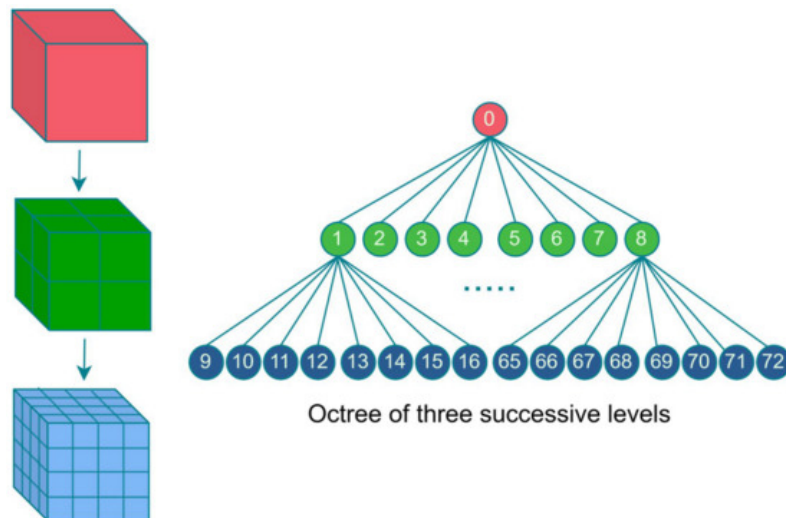


Gambar 1.1 (3D Mesh Objek Sapi)



Gambar 1.2 (Objek Sapi Voxel)

Voxelization merupakan proses konversi objek tiga dimensi yang umum menjadi sebuah model yang tersusun dari kubus-kubus kecil seperti di Minecraft. Proses konversi ini dilakukan dengan memanfaatkan struktur data Octree, yaitu sebuah tree yang setiap percabangan terdiri dari 8 anak. Melalui Octree, sebuah ruang kubus 3D dapat dipotong menjadi 8 kubus berbeda, yang masing-masing dapat dipotong lagi menjadi 8 kubus yang lebih kecil.



Octree of three successive levels

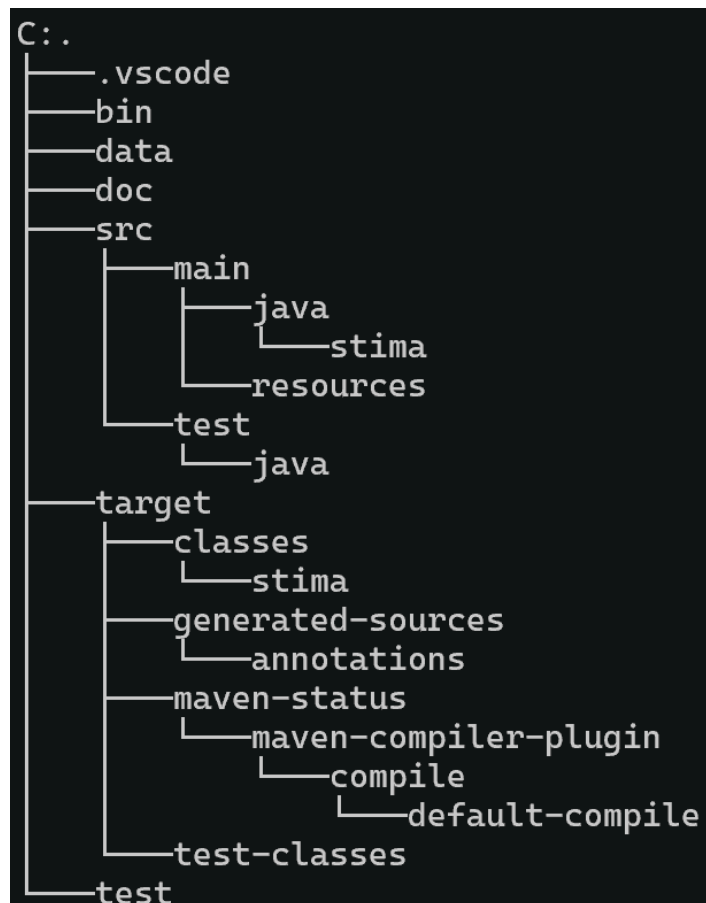
Gambar 1.3 (Visualisasi Octree)

Dalam Tugas Kecil ini kami ditugaskan untuk menghasilkan sebuah program yang dapat melakukan voxelization seperti yang dijelaskan di atas dengan mengimplementasi strategi algoritma divide and conquer.

Divide and Conquer merupakan strategi algoritma dimana kita memecah suatu persoalan menjadi upa-persoalan yang lebih kecil dan memiliki suatu kemiripan. Kemudian masing-masing upa-persoalan itu dilakukan *solve* atau diproses. Setelah diproses, seluruh upa-persoalan digabungkan kembali.

II. Implementasi & Algoritma

Struktur TuCil kami kurang lebih seperti berikut,



Gambar 2.1 (Struktur Repository)

Dengan program utamanya terletak di `src/main/java/stima` berupa file `test.java`. Alur dari programnya kurang lebih berupa *input*, pembuatan octree, dan output.

`test.java`

Proses dimulai dengan pembacaan file OBJ untuk mengumpulkan semua vertex, dilakukan scan pada line yang dimulai dengan huruf 'v' (vertex), lalu parsing untuk koordinat x, y, z dari setiap

vertex. Bersamaan dengan penyimpanan data vertex, nilai minimum dan maksimum pada setiap axis untuk keseluruhan object bounding box akan disimpan.

```
try (Scanner read = new Scanner(file)) {
    while (read.hasNextLine()) {
        String data = read.nextLine().trim();
        if (data.isEmpty() || data.startsWith("#")) {
            continue;
        }
        String[] splitData = data.split("\\s+");
        if (splitData[0].equals("v")) {
            vertex temp = new vertex(
                Double.parseDouble(splitData[1]),
                Double.parseDouble(splitData[2]),
                Double.parseDouble(splitData[3]));
            vertices.add(temp);
            find_extremes(temp); // Update min dan max
        }
    }
}
```

Setelah batas model didapat, program membentuk root cube. Root cube dibuat dengan memilih size terbesar dari batas model (max - min pada setiap sumbu). Kemudian dilakukan pembacaan *faces* dari file, hal ini dilakukan untuk memudahkan iterasi face yang nanti digunakan untuk raycasting.

```
public Tree(vertex max, vertex min, int max_depth) {
    root = new Node(starting_cube(max, min));
}

try (Scanner read = new Scanner(file)) {
    while (read.hasNextLine()) {
        String data = read.nextLine().trim();
        if (data.isEmpty() || data.startsWith("#")) {
            continue;
        }

        String[] splitData = data.split("\\s+");
        if (splitData[0].equals("f")) {
            int a = Integer.parseInt(splitData[1].split("/")[0]);

            // for quads
            for (int i = 2; i < splitData.length - 1; i++) {
                int b = Integer.parseInt(splitData[i].split("/")[0]);
                int c = Integer.parseInt(splitData[i + 1].split("/")[0]);

                face temp = new face(vertices.get(a - 1), vertices.get(b - 1),
```

```

        vertices.get(c - 1));
    faces.add(temp);
    }
}
}
}

```

Bagian raycasting menentukan node mana yang relevan terhadap permukaan mesh. Pada implementasi ini, fungsi raycasting() mengecek apakah sebuah ray memotong atau intersect triangle tertentu menggunakan algoritma Möller-Trumbore, ray direpresentasikan sebagai $\text{origin} + t * \text{direction}$, lalu dicari apakah ada solusi valid terhadap bidang triangle dengan syarat $u \geq 0$, $v \geq 0$, dan $u + v \leq 1$. Jika syarat terpenuhi dan $t > \text{EPSILON}$, titik potong dianggap valid dan koordinat intersection ditambahkan.

```

public static boolean raycasting(vertex origin, vertex direction, face face, vertex intersection) {
    double EPSILON = 0.0000001;

    vertex v1 = face.a;
    vertex v2 = face.b;
    vertex v3 = face.c;

    vertex edge1 = subtract(v2, v1);
    vertex edge2 = subtract(v3, v1);
    vertex h = cross(direction, edge2);
    double a = dot(edge1, h);

    if (a > -EPSILON && a < EPSILON)
        return false;

    double f = 1.0 / a;
    vertex s = subtract(origin, v1);
    double u = f * dot(s, h);
    if (u < 0.0 || u > 1.0)
        return false;

    vertex q = cross(s, edge1);
    double v = f * dot(direction, q);
    if (v < 0.0 || u + v > 1.0)
        return false;

    double t = f * dot(edge2, q);
    if (t > EPSILON) {
        intersection.x = origin.x + direction.x * t;
        intersection.y = origin.y + direction.y * t;
        intersection.z = origin.z + direction.z * t;
        return true;
    }
    return false;
}

```

```
}
```

Fungsi ini digunakan dalam menentukan mana voxel yang aktif dalam suatu face. Sistemnya bekerja dengan menentukan titik asal (origin) dan menembakkan ray ke arah tertentu (direction). Jika ray mengenai bidang face yang dari mesh, akan dihitung koordinat x, y, z perpotongan. Validasi dilakukan di dalam fungsi markActiveVoxel, jika candidateFaces kosong langsung berhenti, lalu jika depth sudah mencapai maxDepth fungsi menghitung titik tengah node dan menembakkan ray ke tiga arah sumbu X, Y, dan Z dengan raycast, jika masih berada di dalam cube node, maka branch voxel dibangun dan cabang itu selesai. Jika depth belum maksimum, menghitung batas tiap 8 child octant, menyaring face kandidat dengan overlap AABB supaya hanya face yang relevan yang diproses, jika childFaces tidak kosong child node dibuat jika belum ada dan fungsi dipanggil rekursif ke level berikutnya, tetapi jika kosong cabang maka skipNodes ditambah.

```
public static void markActiveVoxel(Tree.Node node, ArrayList<face> candidateFaces,
double epsilon,
    double rootMinX, double rootMinY, double rootMinZ, double step, int depth, int
maxDepth) {
    if (candidateFaces.isEmpty()) {
        return;
    }

    if (depth >= maxDepth) {
        vertex center = new vertex(
            (node.surrounding[3].x + node.surrounding[5].x) / 2,
            (node.surrounding[3].y + node.surrounding[5].y) / 2,
            (node.surrounding[3].z + node.surrounding[5].z) / 2);

        for (face f : candidateFaces) {
            vertex intersection = new vertex(0, 0, 0);

            vertex origin = new vertex(rootMinX - step, center.y, center.z);
            vertex dir = new vertex(1, 0, 0);
            if (raycasting(origin, dir, f, intersection)) {
                intersection.x += epsilon;
                if (Tree.is_in_cube(node, intersection)) {
                    ocTree.build_branch(ocTree.root, intersection, 0, maxDepth);
                    return;
                }
            }

            origin = new vertex(center.x, rootMinY - step, center.z);
            dir = new vertex(0, 1, 0);
            if (raycasting(origin, dir, f, intersection)) {
                intersection.y += epsilon;
                if (Tree.is_in_cube(node, intersection)) {
                    ocTree.build_branch(ocTree.root, intersection, 0, maxDepth);
                }
            }
        }
    }
}
```

```

        return;
    }
}

origin = new vertex(center.x, center.y, rootMinZ - step);
dir = new vertex(0, 0, 1);
if (raycasting(origin, dir, f, intersection)) {
    intersection.z += epsilon;
    if (Tree.is_in_cube(node, intersection)) {
        ocTree.build_branch(ocTree.root, intersection, 0, maxDepth);
        return;
    }
}
}
return;
}

double currentLen = node.surrounding[5].x - node.surrounding[3].x;

for (int i = 0; i < 8; i++) {
    vertex[] surroundTheChild = ocTree.octant_vertex(node, i, currentLen);
    ArrayList<face> childFaces = new ArrayList<face>();
    for (face f : candidateFaces) {
        double faceMinX = Math.min(f.a.x, Math.min(f.b.x, f.c.x));
        double faceMinY = Math.min(f.a.y, Math.min(f.b.y, f.c.y));
        double faceMinZ = Math.min(f.a.z, Math.min(f.b.z, f.c.z));
        double faceMaxX = Math.max(f.a.x, Math.max(f.b.x, f.c.x));
        double faceMaxY = Math.max(f.a.y, Math.max(f.b.y, f.c.y));
        double faceMaxZ = Math.max(f.a.z, Math.max(f.b.z, f.c.z));

        // cuz we're not creating the whole tree
        double nodeMinX = surroundTheChild[3].x;
        double nodeMinY = surroundTheChild[3].y;
        double nodeMinZ = surroundTheChild[3].z;
        double nodeMaxX = surroundTheChild[5].x;
        double nodeMaxY = surroundTheChild[5].y;
        double nodeMaxZ = surroundTheChild[5].z;

        if (!(faceMaxX < nodeMinX || faceMinX > nodeMaxX ||
            faceMaxY < nodeMinY || faceMinY > nodeMaxY ||
            faceMaxZ < nodeMinZ || faceMinZ > nodeMaxZ)) {
            childFaces.add(f);
        }
    }

    if (!childFaces.isEmpty()) {
        if (node.children[i] == null) {
            node.children[i] = ocTree.octant_node(node, i, currentLen);
        }
        markActiveVoxel(node.children[i], childFaces, epsilon, rootMinX, rootMinY,

```

```

rootMinZ,
    step, depth + 1, maxDepth);
} else if (depth + 1 < skipNodes.length) {
    skipNodes[depth + 1]++;
}
}
}

```

Daun dari tree dibentuk dengan membentuk Node dari root node Octree sampai max_depth dengan mengecek bagian oktan mana terletak titik *intersect* yang didapatkan dari raycasting. Setiap Node menyimpan informasi child (jika ada) dan titik-titik yang berupa sudut dari sebuah voxel. Satu Node dalam max_depth dari Tree akan merepresentasikan sebuah voxel. Ini akan menghasilkan OcTree hanya dengan Node-node yang akan digunakan untuk pembentukan voxel.

```

public void build_branch(Node node, vertex intersect, int depth, int maxDepth) {
    if (depth >= maxDepth || !is_in_cube(node, intersect)) {
        return;
    }

    double len = node.surrounding[5].x - node.surrounding[3].x;

    for (int i=0;i<8;i++) {
        if (node.children[i] == null) {
            node.children[i] = (new Node(octant_vertex(node, i, len)));
            if (depth + 1 < nodes.length) {
                nodes[depth + 1]++;
            }
        }

        if (is_in_cube(node.children[i], intersect)) {
            build_branch(node.children[i], intersect, depth + 1, maxDepth);
            break;
        }
    }
}

```

Terakhir dilakukan pembentukan file output berupa .obj yang dibantu dengan fungsi build_cubes(). Fungsi ini membentuk string untuk setiap titik/*vertex* dan setiap face dari node yang max_depth milik Tree atau voxelnya.

```

public static void build_cubes(String fileDest, Node node, int depth, int max_depth) {
    if (depth == max_depth) {
        StringBuilder temp = new StringBuilder();
        for (int i = 0; i < 8; i++) {
            temp.append("v " + node.surrounding[i].x + " " + node.surrounding[i].y + " "
                + node.surrounding[i].z + "\n");
        }
        temp.append("f " + (verticesCount + 3) + " " + (verticesCount + 2) + " " +
(verticesCount + 1) + " "
            + verticesCount + "\n");
        temp.append("f " + verticesCount + " " + (verticesCount + 1) + " " + (verticesCount
+ 5) + " "
            + (verticesCount + 4) + "\n");
        temp.append("f " + verticesCount + " " + (verticesCount + 4) + " " + (verticesCount
+ 7) + " "
            + (verticesCount + 3) + "\n");
        temp.append("f " + (verticesCount + 1) + " " + (verticesCount + 2) + " " +
(verticesCount + 6) + " "
            + (verticesCount + 5) + "\n");
        temp.append("f " + (verticesCount + 2) + " " + (verticesCount + 3) + " " +
(verticesCount + 7) + " "
            + (verticesCount + 6) + "\n");
        temp.append("f " + (verticesCount + 4) + " " + (verticesCount + 5) + " " +
(verticesCount + 6) + " "
            + (verticesCount + 7) + "\n");

        try (FileWriter writer = new FileWriter(fileDest, true)) {
            writer.write(temp.toString());
        } catch (IOException e) {
            e.printStackTrace();
        }
        verticesCount += 8;
    } else if (depth < max_depth) {
        for (int i = 0; i < 8; i++) {
            if (node.children[i] != null) {
                build_cubes(fileDest, node.children[i], depth + 1, max_depth);
            }
        }
    }
}

```

viewer3D.java

Selanjutnya adalah implementasi, viewer3D menggunakan javafx canvas. Dimulai dengan fungsi loadObjVoxel() membaca file OBJ voxel lalu menyiapkan vertex untuk dirender. Setelah semua vertex terkumpul, fungsi menghitung bounding box model menggunakan nilai minimum dan maksimum pada sumbu X, Y, dan Z. Dari bounding box, dihitung center dan ukuran terbesar

model (maxSize), lalu semua vertex dinormalisasi dengan $(v[i] - \text{center}) * 2 / \text{maxSize}$, 2 disini adalah arbitrary number untuk menjadi scale factor 1 unit. Proses ini membuat model terpusat serta memiliki skala yang konsisten di canvas. Setelah normalisasi selesai, data dipindahkan ke array vertices, lalu edges dibangun secara manual per 8 vertex sebagai satu kubus voxel dengan total 12 garis edge.

```
private void loadObjVoxel(String path) {
    List<double[]> vertex = new ArrayList<>();
    edges.clear();

    File file = new File(path);

    // create vertices
    try (Scanner read = new Scanner(file)) {
        while (read.hasNextLine()) {
            String data = read.nextLine().trim();
            if (data.isEmpty() || data.startsWith("#")) {
                continue;
            }

            String[] splitData = data.split("\\s+");
            if (splitData[0].equals("v")) {
                double x = Double.parseDouble(splitData[1]);
                double y = Double.parseDouble(splitData[2]);
                double z = Double.parseDouble(splitData[3]);
                vertex.add(new double[] { x, y, z });
            }
        }
    } catch (Exception e) {
        System.err.println("failed");
    }

    // baka centering problem
    // recompute center of the obj file

    double minX = Double.POSITIVE_INFINITY;
    double minY = Double.POSITIVE_INFINITY;
    double minZ = Double.POSITIVE_INFINITY;
    double maxX = Double.NEGATIVE_INFINITY;
    double maxY = Double.NEGATIVE_INFINITY;
    double maxZ = Double.NEGATIVE_INFINITY;
    for (double[] v : vertex) {
        if (v[0] < minX)
            minX = v[0];
        if (v[1] < minY)
            minY = v[1];
        if (v[2] < minZ)
            minZ = v[2];
        if (v[0] > maxX)
```

```

        maxX = v[0];
        if (v[1] > maxY)
            maxY = v[1];
        if (v[2] > maxZ)
            maxZ = v[2];
    }
    double centerX = (minX + maxX) / 2.0;
    double centerY = (minY + maxY) / 2.0;
    double centerZ = (minZ + maxZ) / 2.0;
    double maxSize = Math.max(maxX - minX, Math.max(maxY - minY, maxZ - minZ));

    // recenter all vertex
    for (double[] v : vertex) {
        v[0] = (v[0] - centerX) * 2.0 / maxSize;
        v[1] = (v[1] - centerY) * 2.0 / maxSize;
        v[2] = (v[2] - centerZ) * 2.0 / maxSize;
    }

    // overwrite all vertices
    vertices = vertex.toArray(new double[0][0]);

    if (loadedObjLabel != null) {
        loadedObjLabel.setText("Loaded: " + file.getName());
    }

    // cube edges manual
    for (int i = 0; i + 7 < vertices.length; i += 8) {
        // 12 edges
        edges.add(new int[] { i + 0, i + 1 });
        edges.add(new int[] { i + 1, i + 2 });
        edges.add(new int[] { i + 2, i + 3 });
        edges.add(new int[] { i + 3, i + 0 });
        edges.add(new int[] { i + 4, i + 5 });
        edges.add(new int[] { i + 5, i + 6 });
        edges.add(new int[] { i + 6, i + 7 });
        edges.add(new int[] { i + 7, i + 4 });
        edges.add(new int[] { i + 0, i + 4 });
        edges.add(new int[] { i + 1, i + 5 });
        edges.add(new int[] { i + 2, i + 6 });
        edges.add(new int[] { i + 3, i + 7 });
    }

    // System.out.println("Loaded vertices: " + vertices.length + ", edges: " +
    // edges.size());
}

```

Fungsi `update()` bertugas memproses input keyboard setiap frame. Untuk `keyCode.A`, sudut rotasi `Y` (`angleY`) dikurangi sehingga model berputar ke kiri, dan sebaliknya untuk `keyCode.D`. Untuk zoom, `keyCode.W` menambah `scale` dan `keyCode.S` mengurangi `scale`. Implementasi ini tidak menggunakan kamera, melainkan hanya melakukan perhitungan skala serta proyeksi. Sehingga tidak menggunakan vektor kamera eksplisit.

```
private void update() {
    if (keys.contains(KeyCode.A)) angleY -= 0.03;
    if (keys.contains(KeyCode.D)) angleY += 0.03;

    if (keys.contains(KeyCode.W)) scale += zoomSpeed;
    if (keys.contains(KeyCode.S)) scale -= zoomSpeed;

    if (scale < 50) {
        scale = 50;
    }
}
```

Fungsi `render()` menggambar scene ke canvas berdasarkan state terbaru (`post-update()`). Pertama, clear screen canvas. Seluruh vertex 3D diproyeksikan ke 2D nilai `x` dan `z` dipakai untuk rotasi sumbu `Y`, lalu hasil rotasi dikonversi ke koordinat layar dengan faktor `scale` serta offset layar (`xOffset` dan `yOffset`). Viewer ini memakai pendekatan orthographic sederhana, sehingga tidak ada representasi hasil rotasi untuk sumbu `z` di layar (tidak ada depth), hal ini dilakukan untuk mensimplifikasi viewer serta membuat viewer lebih ringan dalam kompilasi setiap update. Menggunakan `javafx.canvas.GraphicsContext`, terdapat fungsi untuk melakukan draw line antara 2 titik yaitu `gc.strokeLine`. Iterasi setiap pasangan vertex untuk menggambar garis untuk setiap edge.

```
private void render(GraphicsContext gc, double w, double h) {
    gc.setFill(Color.BLACK);
    gc.fillRect(0, 0, w, h);

    double[][] projections = new double[vertices.length][2];
    double xOffset = w / 2.0;
    double yOffset = h / 2.0;

    for (int i = 0; i < vertices.length; i++) {
        double x3D = vertices[i][0];
        double y3D = vertices[i][1];
        double z3D = vertices[i][2];

        double rotatedX = x3D * Math.cos(angleY) - z3D * Math.sin(angleY);
        double x2D = rotatedX * scale + xOffset;
        double y2D = -y3D * scale + yOffset;
```

```

        projections[i][0] = x2D;
        projections[i][1] = y2D;
    }

    gc.setStroke(Color.BLUE);
    gc.setLineWidth(1);
    for (int[] e : edges) {
        int a = e[0];
        int b = e[1];
        if (a < 0 || b < 0 || a >= projections.length || b >= projections.length) continue;
        gc.strokeLine(projections[a][0], projections[a][1], projections[b][0],
projections[b][1]);
    }
}

```

III. Pengujian

Depth is 0-based so 0 is root, we're not doing depth higher than 7 cause the file would be too big to commit to github, though you can try them on your own.

1. Test 1 - cow_6

Input

File: cow.obj

Depth: 6



Output

Created Voxel: 3492

Created Vertex: 27936

Created Faces: 20952

Node Stats:

1 : 4

2 : 20

3 : 91

4 : 384

5 : 1754

6 : 8822

Skipped Nodes Stats:

1 : 4

2 : 12

3 : 69

4 : 344

5 : 1318

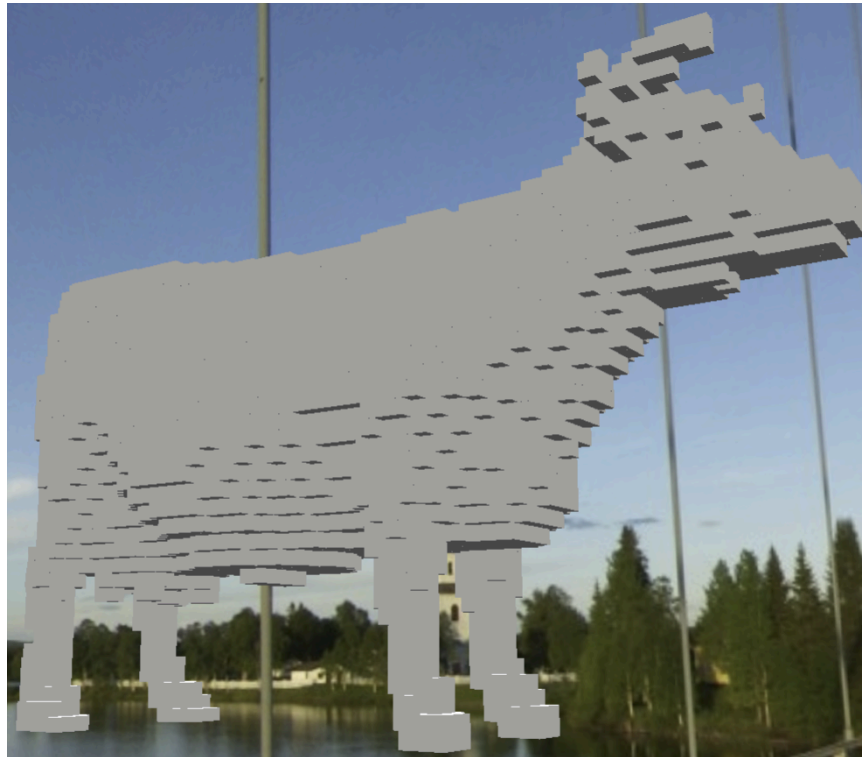
6 : 5210

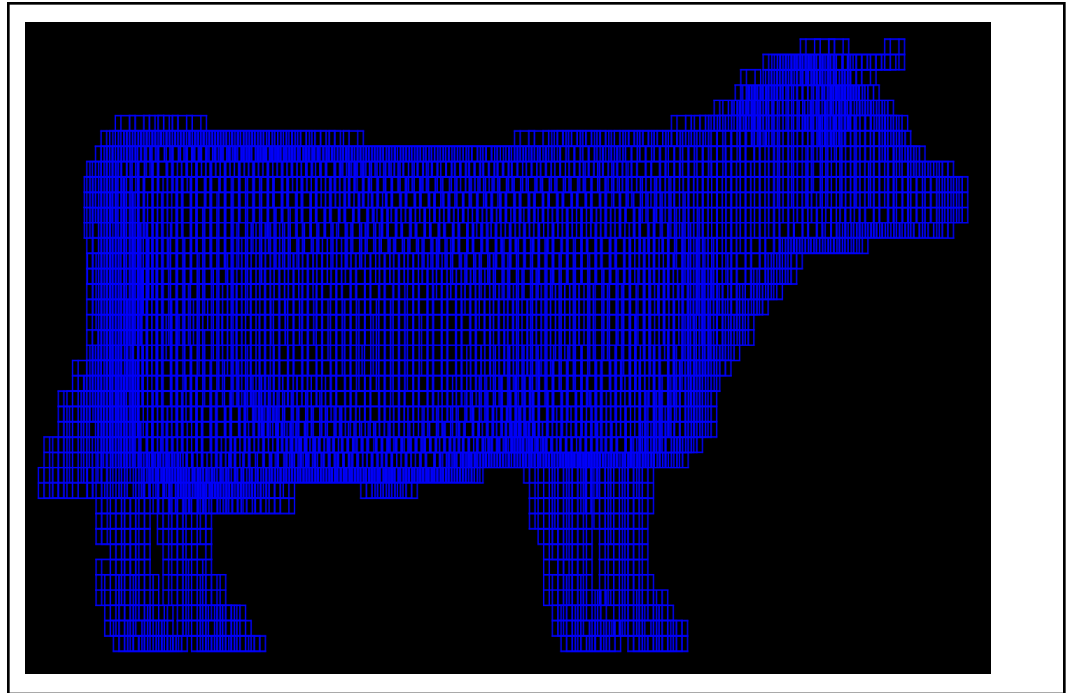
Depth: 6

Time: 207 ms

.obj Saved To:

C:\Users\Alit\Documents\Alit\Shit\Stima\Tucil_2\Tucil_2\.\test\test_1.obj





2. Test 2 - cow_7

Input

File: cow.obj

Depth: 7



Output

Created Voxel: 14112

Created Vertex: 112896

Created Faces: 84672

Node Stats:

1 : 4

2 : 20

3 : 91

4 : 384

5 : 1754

6 : 8822

7 : 48645

Skipped Nodes Stats:

1 : 4

2 : 12

3 : 69

4 : 344

5 : 1318

6 : 5210

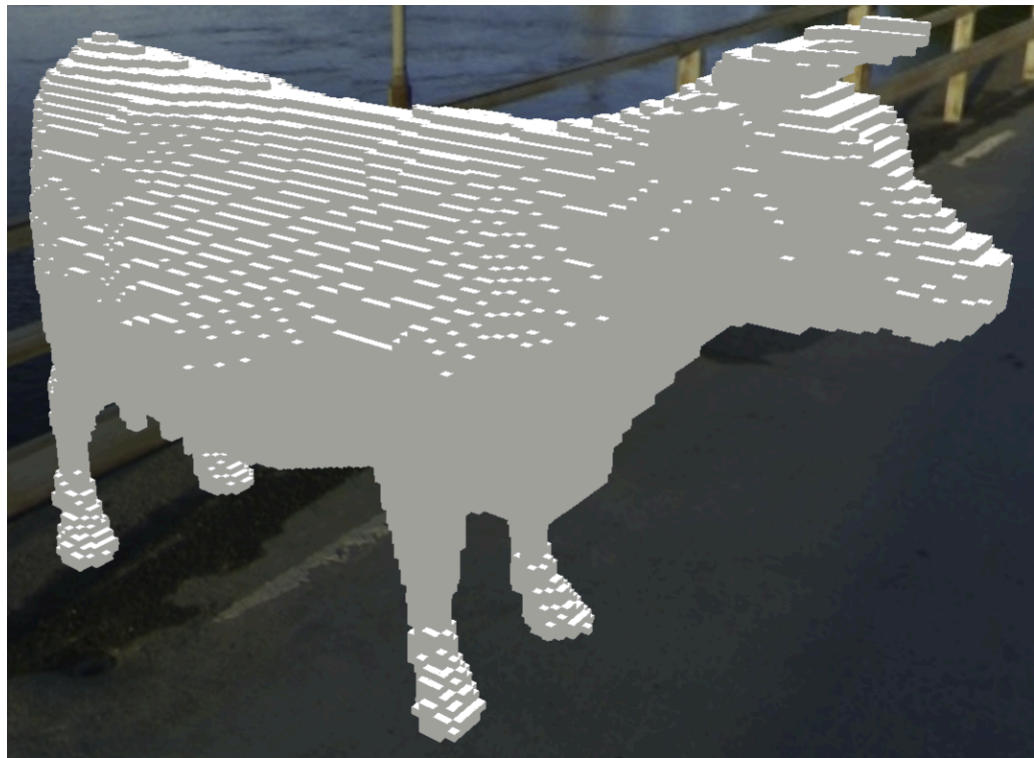
7 : 21931

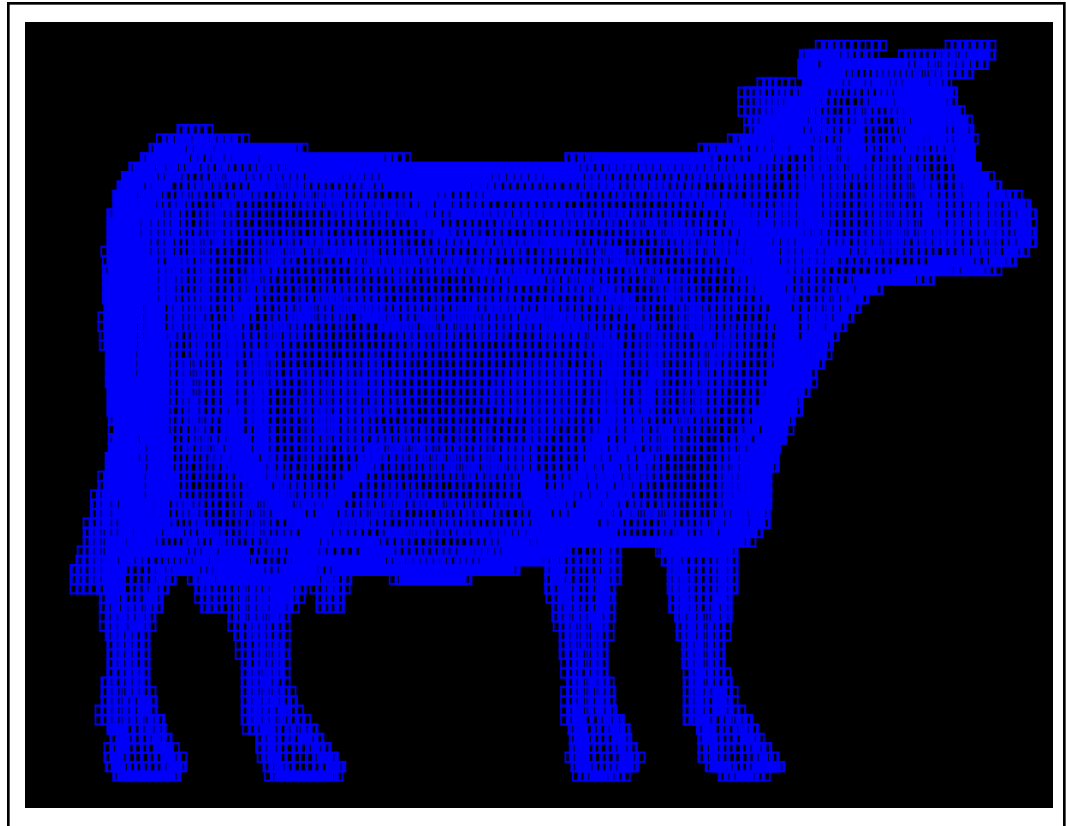
Depth: 7

Time: 148 ms

.obj Saved To:

C:\Users\Alit\Documents\Alit\Shit\Stima\Tucil_2\Tucil_2\.\test\test_2.obj





3. Test 3 - cow_1

Input

File: cow.obj

Depth: 1



Output

Created Voxel: 2

Created Vertex: 16

Created Faces: 12

Node Stats:

1 : 4

Skipped Nodes Stats:

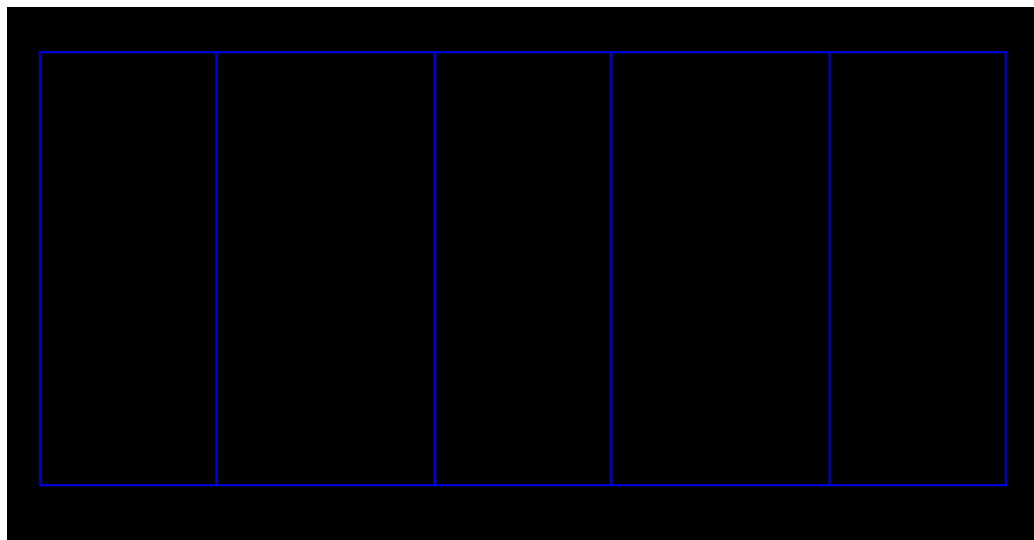
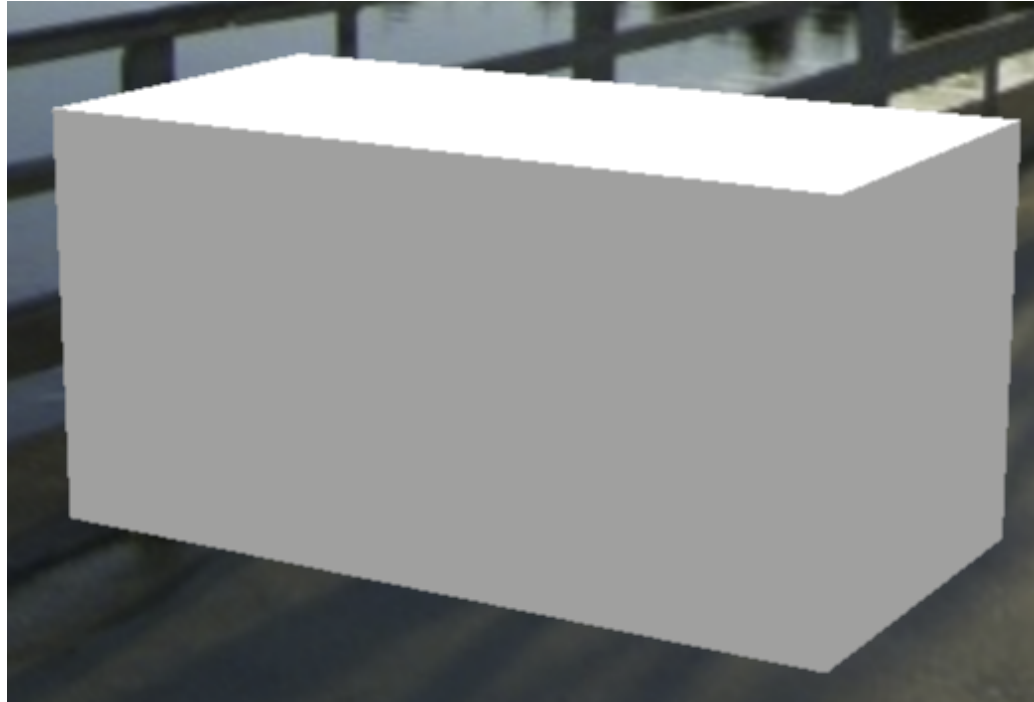
1 : 4

Depth: 1

Time: 24 ms

.obj Saved To:

C:\Users\Alit\Documents\Alit\Shit\Stima\Tucil_2\Tucil_2\.\test\test_3.obj

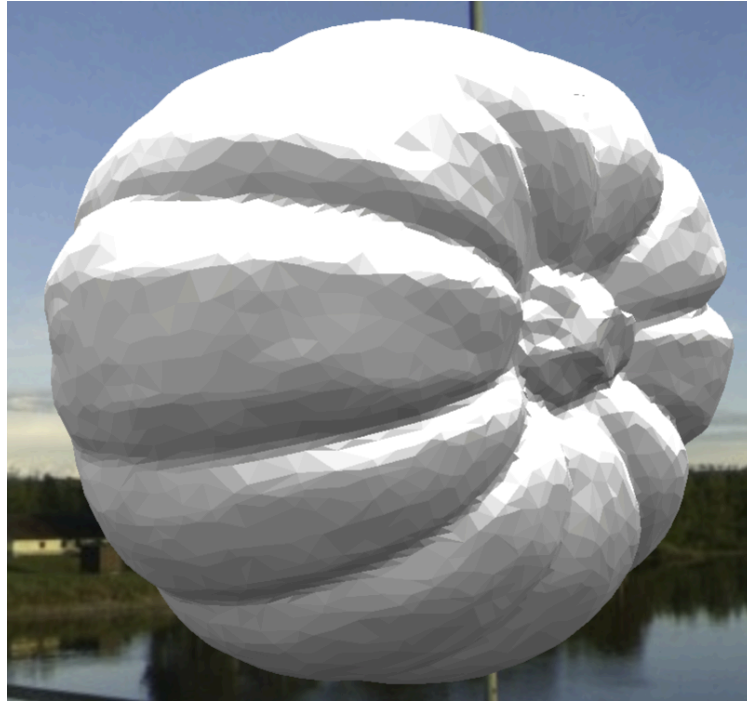


4. Test 4 - pumpkin_6

Input

File: pumpkin.obj

Depth: 6



Output

Created Voxel: 10573

Created Vertex: 84584

Created Faces: 63438

Node Stats:

1 : 8

2 : 58

3 : 274

4 : 1315

5 : 6366

6 : 33045

Skipped Nodes Stats:

1 : 0

2 : 6

3 : 190

4 : 877

5 : 4154

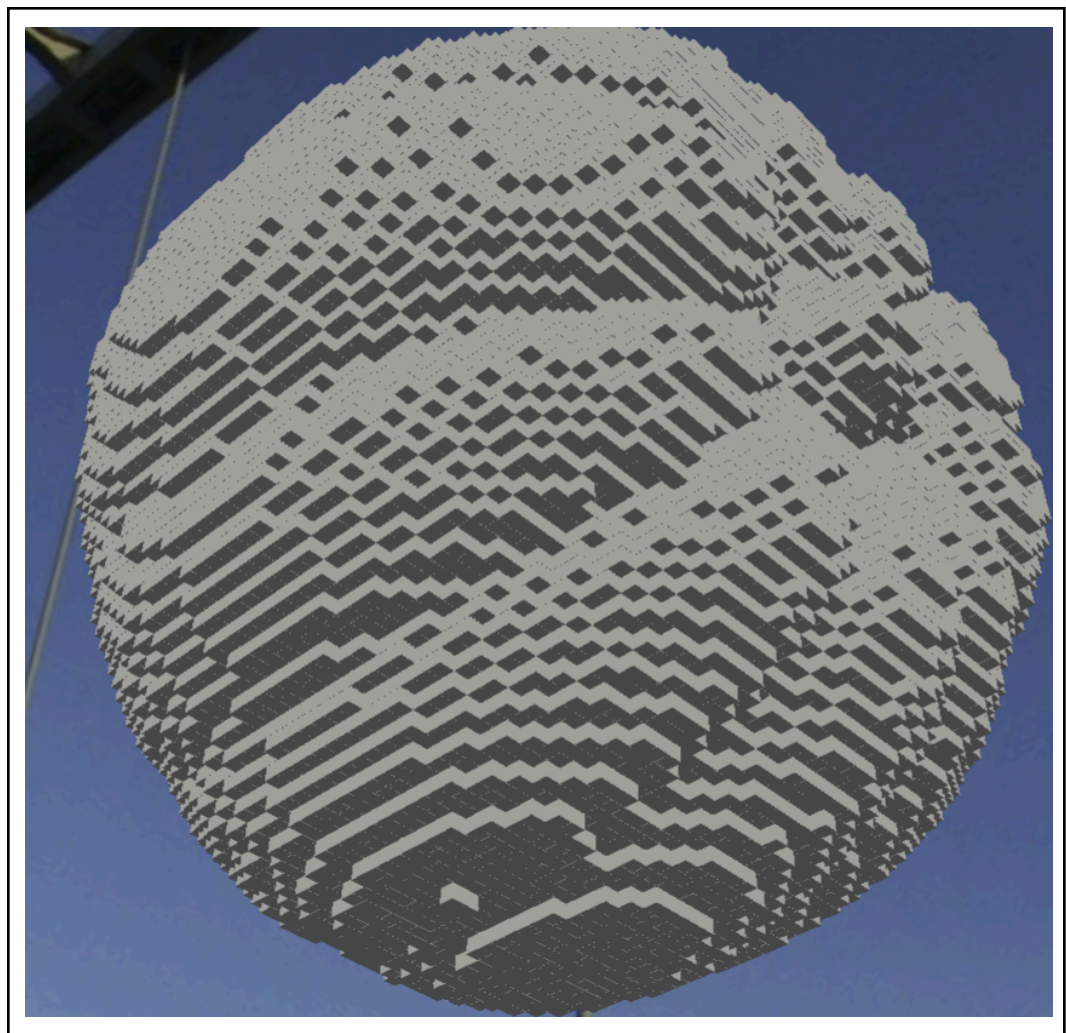
6 : 17883

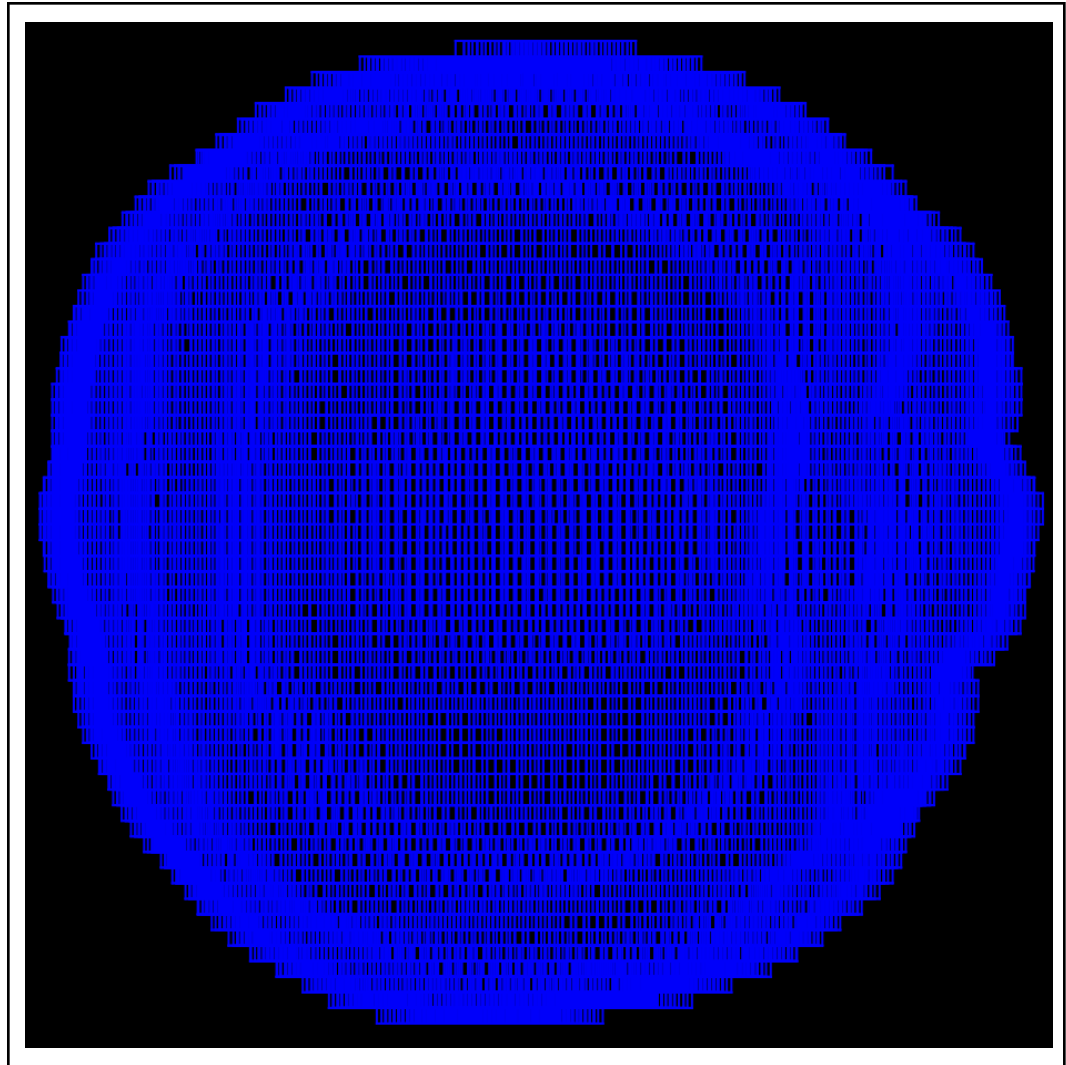
Depth: 6

Time: 140 ms

.obj Saved To:

C:\Users\Alit\Documents\Alit\Shit\Stima\Tucil_2\Tucil_2\.\test\test_4.obj



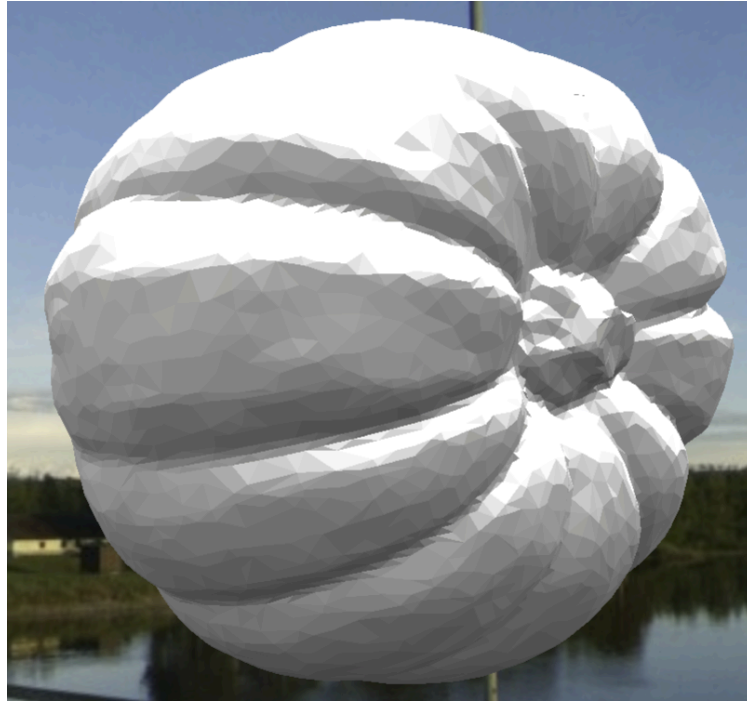


5. Test 5 - pumpkin_3

Input

File: pumpkin.obj

Depth: 3



Output

Created Voxel: 163

Created Vertex: 1304

Created Faces: 978

Node Stats:

1 : 8

2 : 58

3 : 274

Skipped Nodes Stats:

1 : 0

2 : 6

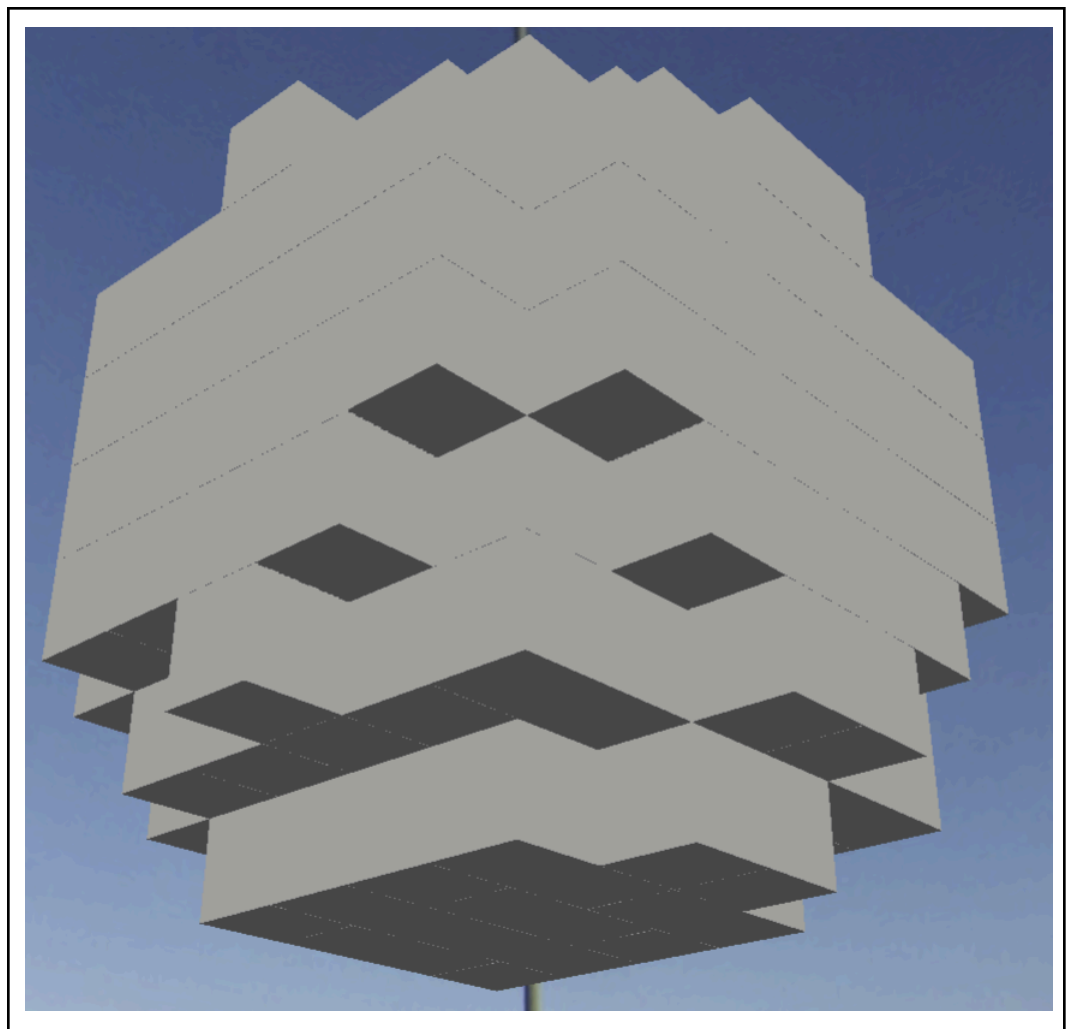
3 : 190

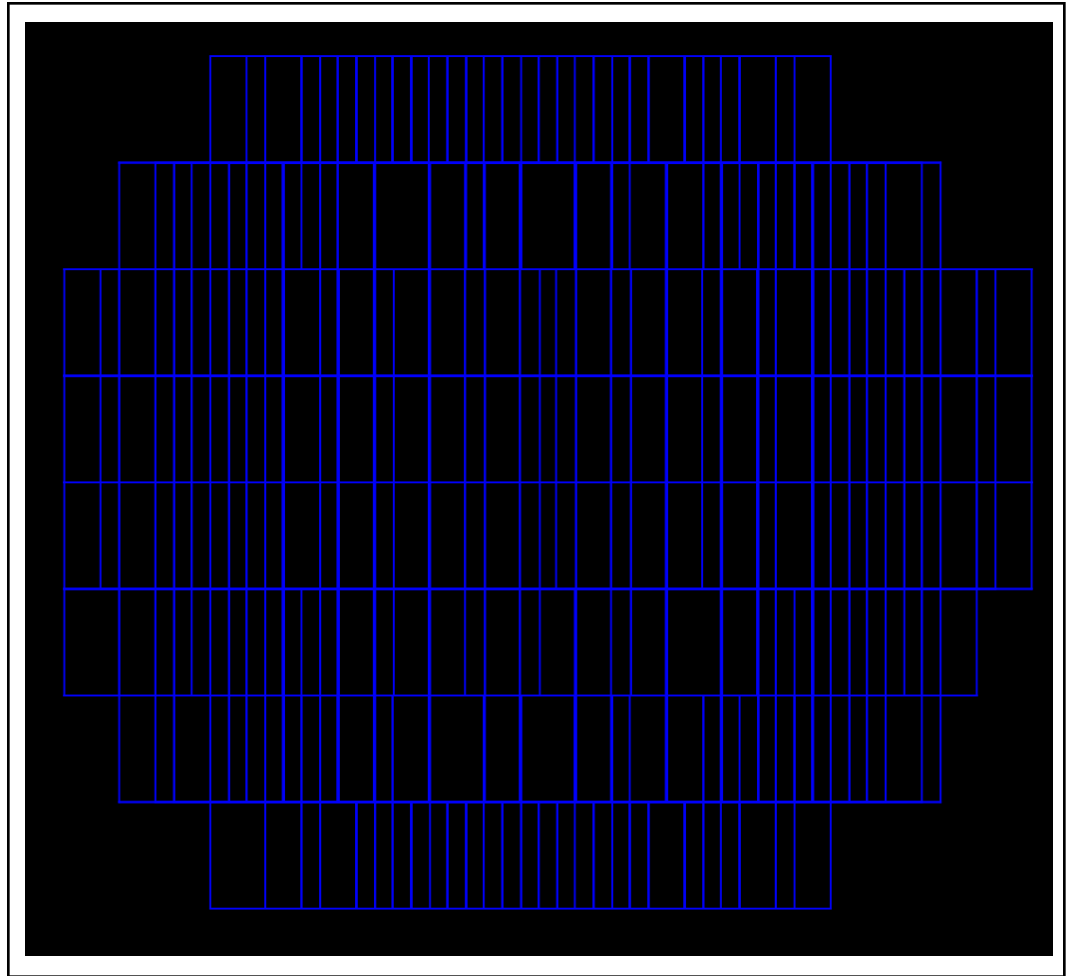
Depth: 3

Time: 40 ms

.obj Saved To:

C:\Users\Alit\Documents\Alit\Shit\Stima\Tucil_2\Tucil_2\.\test\test_5.obj



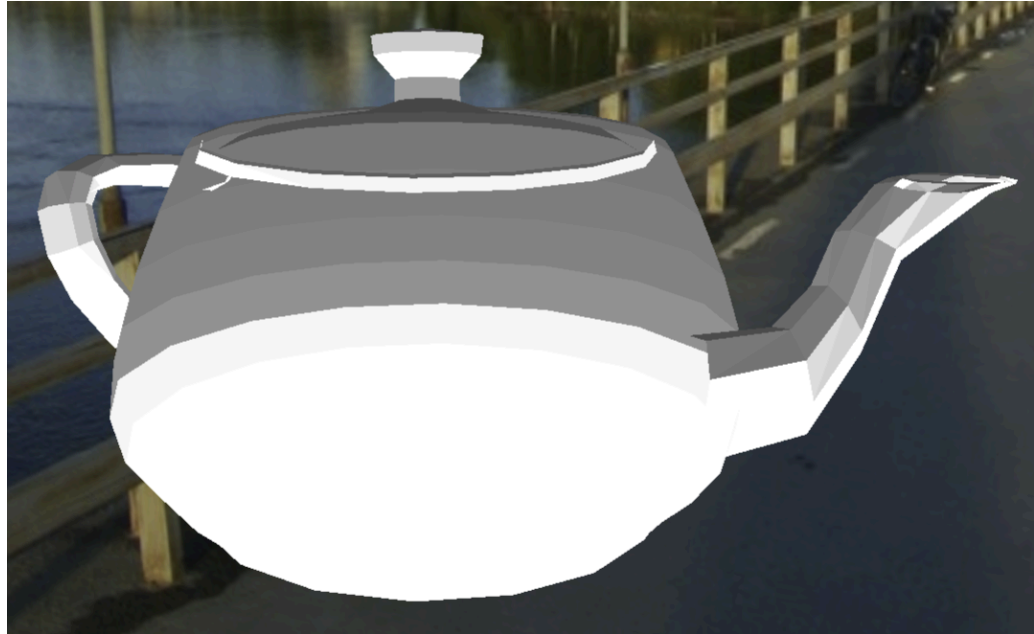


6. Test 6 - teapot_7

Input

File: teapot.obj

Depth: 7



Output

Created Voxel: 17541

Created Vertex: 140328

Created Faces: 105246

Node Stats:

1 : 4

2 : 23

3 : 110

4 : 560

5 : 2973

6 : 17589

7 : 114240

Skipped Nodes Stats:

1 : 4

2 : 9

3 : 74

4 : 320

5 : 1507

6 : 6195

7 : 26472

Depth: 7

Time: 145 ms

.obj Saved To:

C:\Users\Alit\Documents\Alit\Shit\Stima\Tucil_2\Tucil_2\.\test\test_6.obj



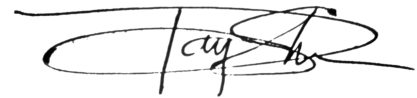
IV. Lampiran

[Repository Github](#)

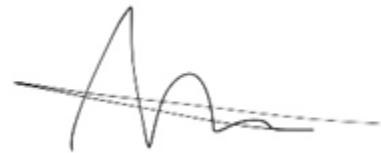
No	Poin	Ya	Tidak
1	Program berhasil di kompilasi tanpa kesalahan	1	
2	Program berhasil dijalankan	1	

3	Hasil konversi .obj terdiri dari kubus-kubus kecil	1	
4	Program informasi-informasi atau report	1	
5	Program dapat menerima masukan .obj, dan mengembalikan output	1	
6	[Bonus] Program menggunakan <i>concurrency</i>		1
7	[Bonus] Program dapat menampilkan visualisasi .obj	1	

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.



Raysha Erviandika Putra



I Gusti Ngurah Alit Dharma Yudha

Extras:



Absolute spiki
- Alit



Gogogaga...

- Raysha